Data Cleaning dan Quality di Skala Besar

Pertemuan 4 — Analisis Big Data

S1 Teknik Informatika / Sistem Informasi | Semester Ganjil 2026

Agenda Pertemuan 4

1. 6 Dimensi Kualitas Data
2. Profiling cepat dengan DuckDB `SUMMARIZE`
3. Menangani missing values dengan Polars
4. Deteksi dan penanganan outlier (IQR + domain knowledge)
5. Standardisasi teks dan validasi email
6. Menangani duplikat
7. Great Expectations: validasi data otomatis
8. Pipeline cleaning yang modular dan reproducible

Milestone 2 Tugas 1 diumumkan hari ini! Target check-in: Pertemuan 5

6 Dimensi Kualitas Data

Dimensi	Pertanyaan Kunci	Contoh Masalah
Completeness	Apakah semua nilai yang seharusnya ada, tersedia?	Kolom rating kosong (null)
Accuracy	Apakah nilai sesuai kondisi nyata?	Harga tercatat negatif
Consistency	Apakah format seragam di seluruh data?	"jakarta", "JAKARTA", " Jakarta "
Timeliness	Apakah data cukup baru untuk dipakai?	Data stok yang sudah usang
Uniqueness	Apakah tidak ada duplikasi yang tidak disengaja?	Baris transaksi tercatat dua kali
Validity	Apakah nilai sesuai aturan/domain yang disepakati?	Email tanpa simbol `@`

> Sebelum membersihkan data, tentukan dulu dimensi mana yang paling bermasalah — strategi cleaning berbeda untuk tiap dimensi.

Profiling Cepat dengan DuckDB `SUMMARIZE`

Satu perintah untuk mendapatkan ringkasan statistik seluruh kolom: tipe data, null%, min/max, distinct count, dan lainnya.

```
import duckdb

con = duckdb.connect()
con.register("transaksi", df_kotor)  # DataFrame Polars langsung di-query

profil = con.execute("SUMMARIZE SELECT * FROM transaksi").df()
print(profil)
```

```
# Laporan kualitas data kustom dengan SQL
con.execute("""
    SELECT
        COUNT(*) AS total_baris,
        COUNT(*) - COUNT(DISTINCT id_transaksi) AS duplikat,
        SUM(CASE WHEN harga <= 0 THEN 1 ELSE 0 END) AS harga_invalid
    FROM transaksi
""").df()
```

Menangani Missing Values

Strategi bergantung pada persentase missing, jenis data, dan dampaknya pada analisis.

```
df_cleaned = df_kotor.with_columns([
    # Numerik: isi dengan median
    pl.col("kuantitas").fill_null(pl.col("kuantitas").median()),

    # Numerik kontekstual: isi dengan mean per grup
    pl.col("rating").fill_null(
        pl.col("rating").mean().over("kategori")
    ).round(1),

    # Kategorik: isi dengan nilai default/modus
    pl.col("status").fill_null("proses"),
])
```

> Hati-hati: mengisi dengan mean/median bisa menyesatkan jika distribusi data sangat miring atau terdapat outlier ekstrem.

Deteksi Outlier: Konsep Interquartile Range (IQR)

Metode berbasis kuartil yang **robust** (kebal nilai ekstrem) untuk mendeteksi anomali pada distribusi data numerik, terutama data yang miring (*skewed*).

Komponen Kuartil:

- **Q1 (Kuartil 1 / P25):** 25% data berada di bawah nilai ini.
- **Q2 (Median / P50):** Titik tengah data terurut.
- **Q3 (Kuartil 3 / P75):** 75% data berada di bawah nilai ini.
- **IQR = Q3 - Q1:** Rentang sebaran 50% data di area tengah (*middle 50%*).

Aturan Tukey's Fences (Batas Outlier):

- **Batas Bawah:** `Q1 - 1.5 * IQR`
- **Batas Atas:** `Q3 + 1.5 * IQR`
- Nilai di luar rentang `[Batas Bawah, Batas Atas]` diidentifikasi sebagai **potensial outlier** (seperti *whiskers* pada Boxplot).
- Faktor **1.5** mencakup ±99.3% data pada distribusi normal.

> **Kenapa IQR, bukan Z-Score (Mean ± Std)?** Rata-rata (*mean*) dan deviasi standar mudah terdistorsi oleh outlier itu sendiri, sedangkan median dan kuartil bersifat resisten (*outlier-resistant*).

Penanganan Outlier: Polars & Domain Knowledge

1. Hitung Batas IQR dengan Polars

```
q1 = df["harga"].quantile(0.25)
q3 = df["harga"].quantile(0.75)
iqr = q3 - q1

batas_bawah = q1 - 1.5 * iqr
batas_atas = q3 + 1.5 * iqr
```

2. Tiga Pilihan Aksi:

- **Drop/Filter:** Hapus jika data tidak logis (`harga <= 0`).
- **Winsorize (Clip):** Pangkas ke batas wajar.
- **Flagging:** Tandai baris dengan kolom boolean `is_outlier`.

Kombinasi dengan Domain Knowledge:

```
# Batas bisnis: harga e-commerce wajar
# Rp1.000 s/d Rp50.000.000
df_clean = (
    df.filter(pl.col("harga") > 0)
    .with_columns(
        pl.col("harga")
        .clip(
            lower_bound=1_000,
            upper_bound=50_000_000,
        )
        .alias("harga_clean")
    )
)
```

> **Prinsip Penting:** IQR matematis murni berisiko salah membuang transaksi grosir/B2B bernilai besar yang sah. Selalu validasi batas statistik dengan pengetahuan bisnis nyata sebelum memutuskan *drop* atau *clip*.

Standardisasi Teks dan Validasi Email

```
df_std = df.with_columns([
    # Hapus spasi ekstra, seragamkan Title Case
    pl.col("kota").str.strip_chars().str.to_titlecase(),
    pl.col("kategori").str.strip_chars().str.to_titlecase(),
    pl.col("status").str.strip_chars().str.to_lowercase(),

    # Validasi format email dengan regex
    pl.col("email_pembeli")
    .str.contains(r"^[a-zA-Z0-9._%+\-]+@[a-zA-Z0-9.\-]+\.[a-zA-Z]{2,}$")
    .alias("email_valid"),
])

# Email tidak valid diubah menjadi null (anonim), bukan dihapus barisnya
df_std = df_std.with_columns(
    pl.when(pl.col("email_valid")).then(pl.col("email_pembeli")).otherwise(None)
    .alias("email_pembeli")
)
```

Menangani Duplikat

```
total_baris = len(df)
baris_unik = df.n_unique()
print(f"Duplikat: {total_baris - baris_unik:,}")

# Hapus duplikat berdasarkan kunci, pertahankan kemunculan pertama
df_dedup = df.unique(
    subset=["id_transaksi"],
    keep="first",
    maintain_order=True,
)
```

> Deduplikasi harus berdasarkan **kolom kunci bisnis** (misalnya `id_transaksi`), bukan seluruh baris — dua transaksi berbeda bisa saja punya nilai kolom lain yang kebetulan sama.

Great Expectations: Validasi Data Otomatis

Mendefinisikan aturan kualitas data secara deklaratif agar bisa dicek otomatis di pipeline produksi.

```
import great_expectations as gx

ctx = gx.get_context(mode="ephemeral")
suite = ctx.suites.add(gx.ExpectationSuite(name="aturan_transaksi"))

suite.add_expectation(
    gx.expectations.ExpectColumnValuesToBeBetween(
        column="harga_clean", min_value=1_000, max_value=50_000_000
    )
)
suite.add_expectation(
    gx.expectations.ExpectColumnValuesToBeUnique(column="id_transaksi")
)
suite.add_expectation(
    gx.expectations.ExpectColumnValuesToNotBeNull(column="id_transaksi")
)
```

> Jika `great_expectations` tidak terpasang, aturan yang sama dapat ditulis manual dengan `CASE WHEN` di DuckDB sebagai alternatif.

Pipeline Cleaning yang Modular

Gabungkan seluruh langkah menjadi satu fungsi yang terdokumentasi dan dapat direproduksi.

```
def pipeline_data_cleaning(df_input: pl.DataFrame) -> pl.DataFrame:
    return (
        df_input
        .unique(subset=["id_transaksi"], keep="first", maintain_order=True)
        .filter(pl.col("harga").is_not_null() & (pl.col("harga") > 0))
        .with_columns([
            pl.col("kuantitas").fill_null(pl.col("kuantitas").median()),
            pl.col("rating").fill_null(pl.col("rating").mean().over("kategori")).round(1),
            pl.col("status").fill_null("proses"),
        ])
        .with_columns([
            pl.col("kota").str.strip_chars().str.to_titlecase(),
            pl.col("harga").clip(lower_bound=1_000, upper_bound=50_000_000),
        ])
    )

df_bersih = pipeline_data_cleaning(df_kotor)
```

Ringkasan Pertemuan 4

Hari ini kita telah mempelajari:

- **6 Dimensi Kualitas Data**: Completeness, Accuracy, Consistency, Timeliness, Uniqueness, Validity
- **Profiling cepat** dengan DuckDB `SUMMARIZE` untuk gambaran menyeluruh dalam satu perintah
- **Missing values**: hapus, isi median/mean/modus, atau isi berdasarkan konteks grup
- **Outlier**: metode IQR dikombinasikan dengan domain knowledge dan windsorization
- **Standardisasi teks** dan validasi format (regex email)
- **Deduplikasi** dan **Great Expectations** untuk validasi otomatis

Milestone 2 Tugas 1: Data Quality dan Profiling — DuckDB `SUMMARIZE` + transformasi Polars.
Target Check-in: Pertemuan 5.

Pertemuan Berikutnya: Exploratory Data Analysis (EDA) dan Visualisasi Interaktif.

Referensi

- Polars User Guide — Missing Data: https://docs.pola.rs/user-guide/expressions/missing-data/
- DuckDB `SUMMARIZE`: https://duckdb.org/docs/sql/statements/summarize.html
- Great Expectations Documentation: https://docs.greatexpectations.io/